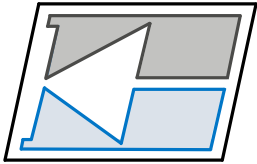


INSTITUTO FEDERAL
Santa Catarina



Departamento Acadêmico de Eletrônica – **DAELN**
IFSC Câmpus Florianópolis

Programação C++

Operações *bitwise*

Prof. Matheus Leitzke Pinto
matheus.pinto@ifsc.edu.br

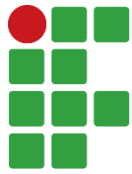
Tipos de dados compostos

- Operações ***bitwise*** (bit-a-bit) são operações que manipulam bits individuais dentro de um número.
- Essas operações são essenciais em programação de baixo nível, como em sistemas embarcados, manipulação de hardware, criptografia, compressão de dados, entre outros.

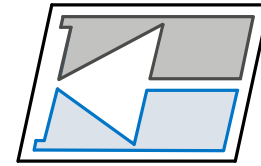
Sumário de aula

- NOT *bitwise*
- AND *bitwise*
- OR *bitwise*
- Deslocamento de bits





INSTITUTO FEDERAL
Santa Catarina



DAELN

Operações *bitwise*

NOT *bitwise*

NOT *bitwise*

- Toda variável é representada por um conjunto de bits.
- Tomemos o exemplos a seguir:

char c = 48;

Representação na memória:

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

NOT bitwise

- Toda variável é representada por um conjunto de bits.
- Tomemos o exemplos a seguir:

Representação de
uma constante em
binário direto.

char c = 0b00110000;

Representação na memória:

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

NOT *bitwise*

- A operação **NOT *bitwise*** tem como função realizar a negação lógica de cada bit dentro de um valor.
- Tomemos o exemplos a seguir:

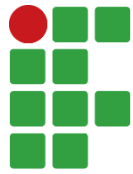
Representação na memória:

char c = **0b**00110000;

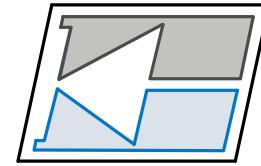
0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

char c = ~**0b**00110000;

1	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---



INSTITUTO FEDERAL
Santa Catarina



DAELN

Operações *bitwise*

OR *bitwise*

OR *bitwise*

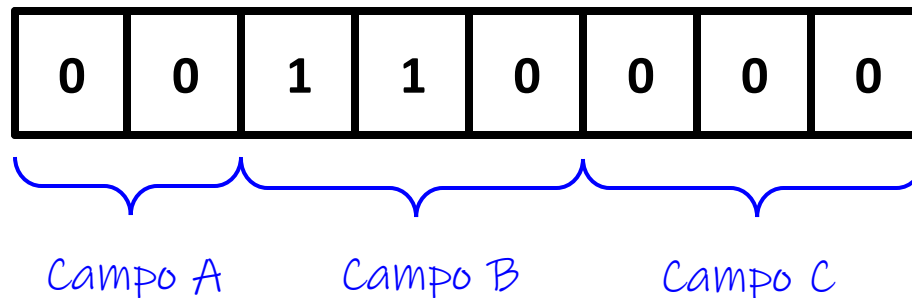
- É comum em determinadas aplicações, que apenas alguns bits específicos de uma variável sejam **setados** (colocados em nível lógico 1.)
- Para tanto, geralmente utiliza-se a operação **OR *bitwise***, em conjunto com uma **máscara de bits**.
- Uma máscara de bits é uma constante quem tem por função ser aplicada à variável, de forma a modificar apenas os bits desejados.

OR bitwise

- Considere o exemplo em que uma variável de 8 bits possua diferentes campos.
- Cada campo, representa uma informação separada.

```
char c = 0b00110000;
```

Representação na memória:

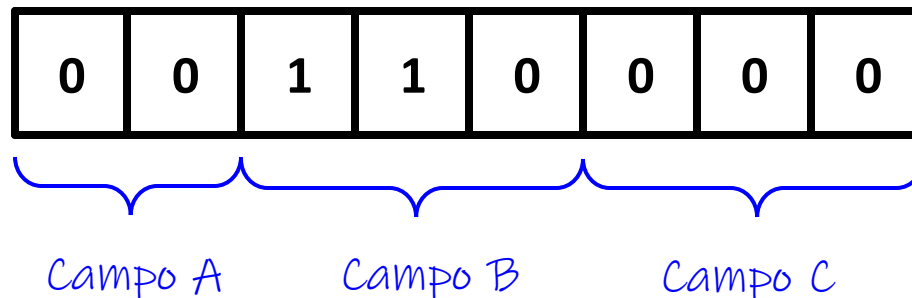


OR bitwise

- Agora vamos considerar que queremos colocar o valor lógico 1 no bit mais a direita do campo A.
- Por outro lado, não queremos modificar o valor dos outros bits.

```
char c = 0b00110000;
```

Representação na memória:

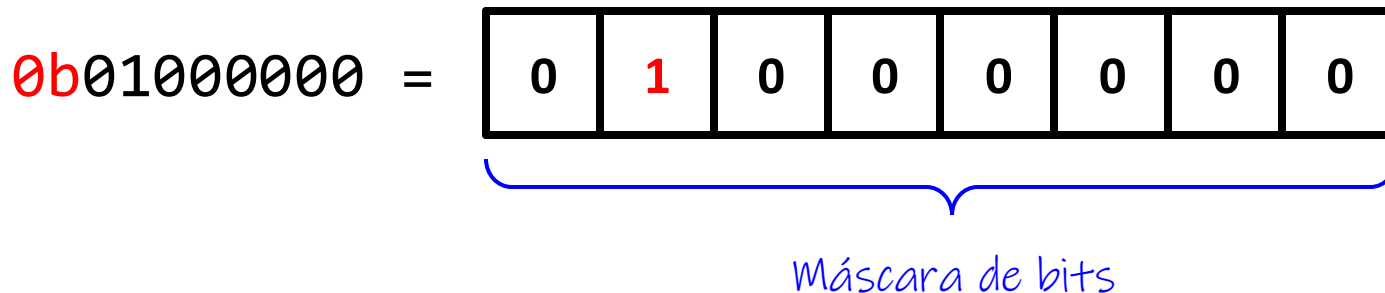
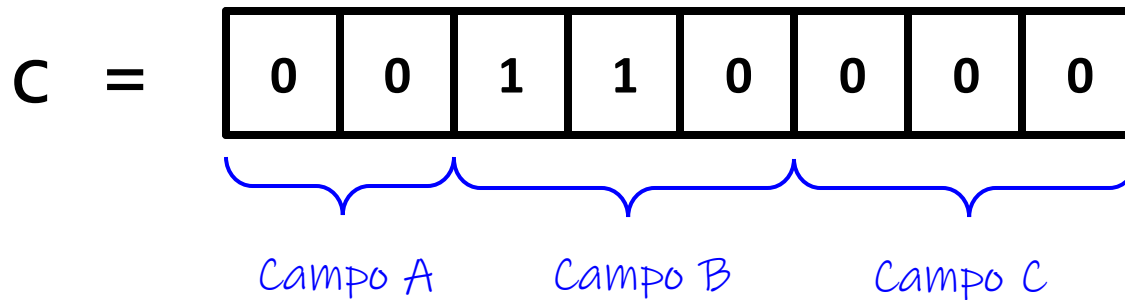


OR bitwise

- Tentativa 1 - Faremos o seguinte:

```
char c = 0b00110000;
```

Representação na memória:



OR bitwise

char c = 0b00110000;

- Tentativa 1 - Faremos o seguinte:

Representação na memória:

C =

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

0b01000000 =

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

c = 0b01000000

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

Com a operação de atribuição simples, setamos os bits que a gente queria, mas também limpamos os bits que não queríamos...

OR bitwise

`char c = 0b00110000;`

- Tentativa 2 - Faremos o seguinte:

Representação na memória:

`c` =

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

`0b01000000` =

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

`c = c | 0b01000000`

0	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

OR
bitwise

OR bitwise

`char c = 0b00110000;`

- Tentativa 2 - Faremos o seguinte:

Representação na memória:

`c` =

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

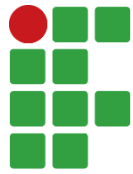
`0b01000000` =

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

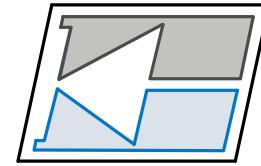
`c |= 0b01000000`

0	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

 OR
bitwise



INSTITUTO FEDERAL
Santa Catarina



DAELN

Operações *bitwise*

AND *bitwise*

AND *bitwise*

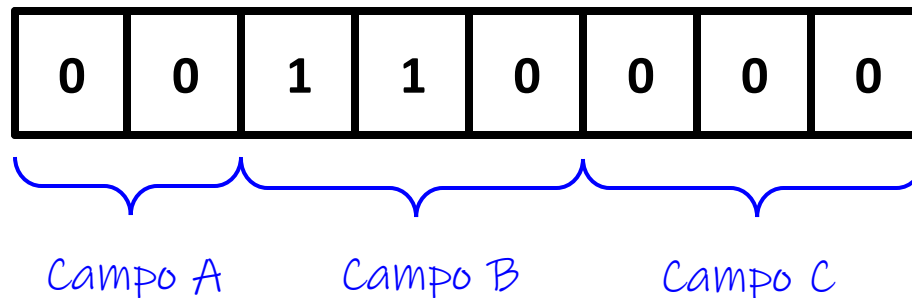
- Enquanto a operação OR *bitwise* é geralmente utilizada em conjunto com uma máscara de bits para setar determinados bits, a operação **AND *bitwise*** é utilizada pra zerar determinados bits.
- Também utilizamos uma máscara de bits, mas inversa ao que vimos em operações OR *bitwise*.

AND bitwise

- Considere o exemplo em que uma variável de 8 bits possua diferentes campos.
- Cada campo, representa uma informação separada.

```
char c = 0b00110000;
```

Representação na memória:

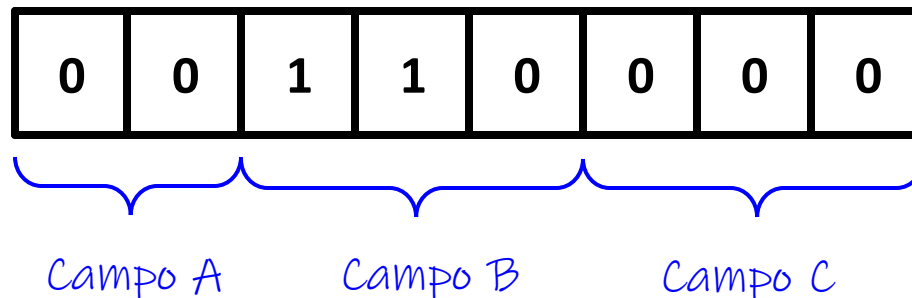


AND bitwise

- Agora vamos considerar que queremos colocar o valor lógico 0 no bit mais a esquerda do campo B.
- Por outro lado, não queremos modificar o valor dos outros bits.

```
char c = 0b00110000;
```

Representação na memória:



AND bitwise

`char c = 0b00110000;`

- Faremos o seguinte:

Representação na memória:

`c` =

0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

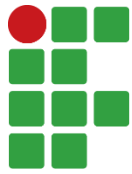
`0b11011111` =

1	1	0	1	1	1	1	1
---	---	---	---	---	---	---	---

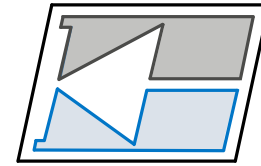
`c &= 0b11011111`

0	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---

AND
bitwise



INSTITUTO FEDERAL
Santa Catarina



DAELN

Operações *bitwise*

Deslocamento de bits

Deslocamento de bits

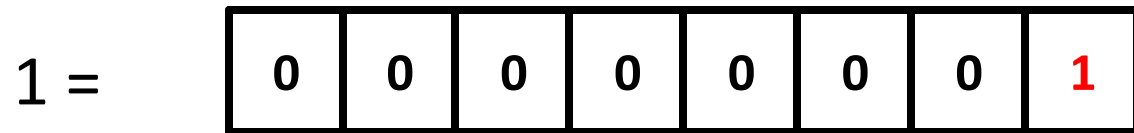
- O deslocamento de bits é uma operação que move os bits de um valor para a esquerda ou para a direita.
- Em C++, isso pode ser feito usando os operadores << (deslocamento para a esquerda) e >> (deslocamento para a direita).

Deslocamento de bits

- O operador de deslocamento para a esquerda (<<) move os bits de um valor para a esquerda por um número especificado de posições.
- Os bits "deslocados" para fora da extremidade esquerda são descartados e os novos bits que entram pelo lado direito são preenchidos com zeros.

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

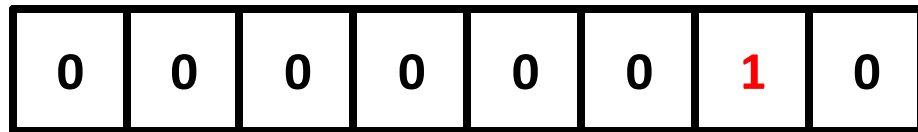


Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$1 \ll 1 =$

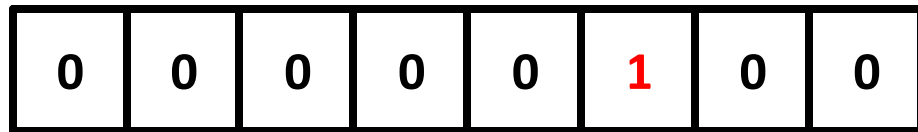


Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$1 \ll 2 =$



Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$$1 \ll 3 =$$

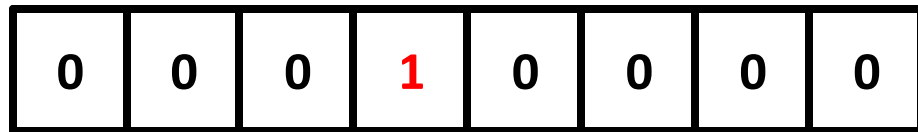
0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$1 \ll 4 =$



Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$$1 \ll 5 =$$

0	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---

Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$$1 \ll 6 =$$

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$$1 \ll 7 =$$

1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

Começamos com o valor decimal 1

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

$$1 \ll 8 =$$

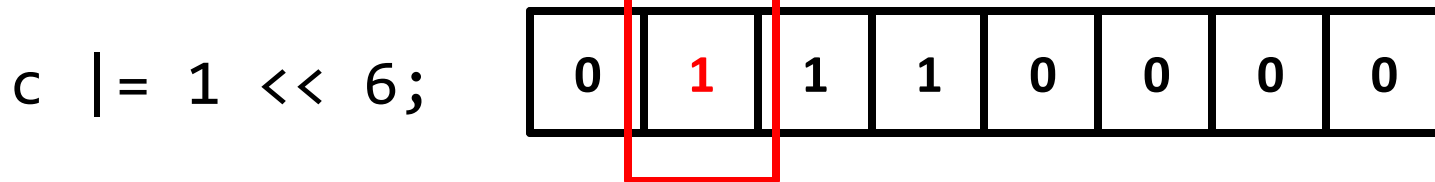
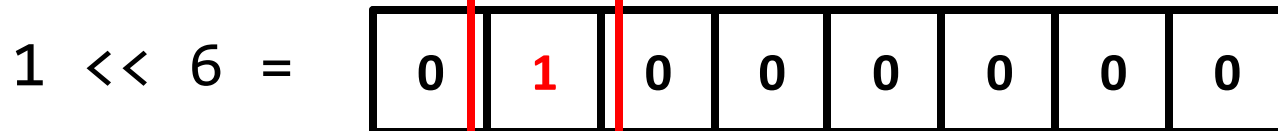
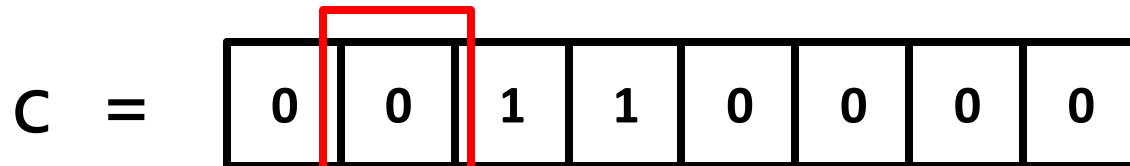
0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

Começamos com o valor decimal 1

Deslocamento de bits

- Usando deslocamento bits para criação de máscara

Representação na memória:



OR
bitwise

Deslocamento de bits

- Aplicando negação:

$$1 \ll 5 =$$

0	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---

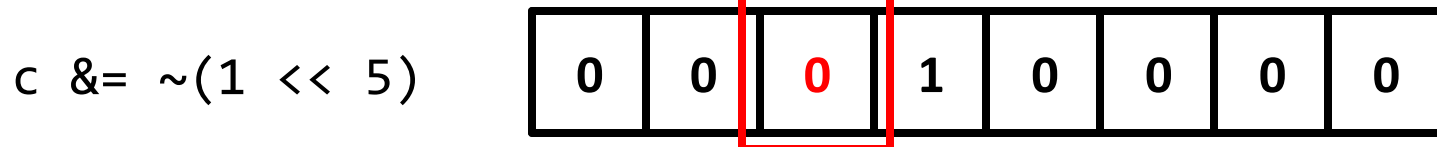
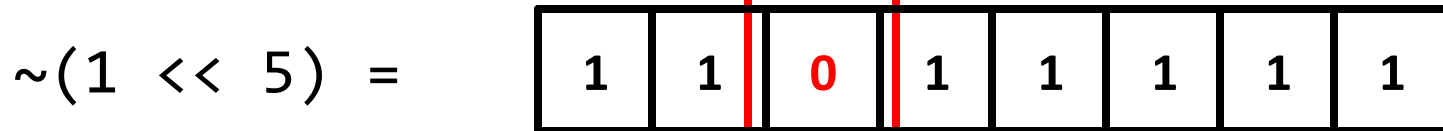
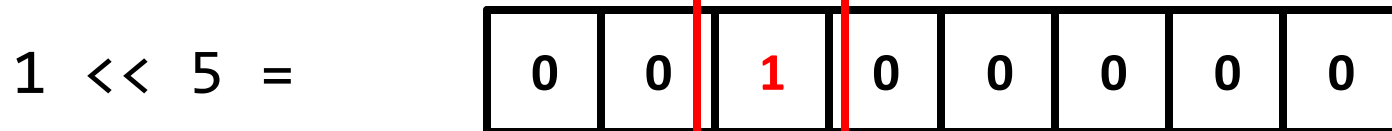
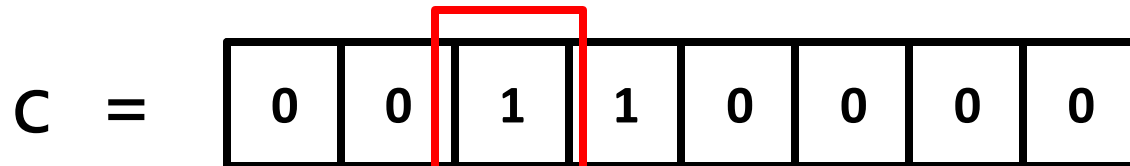
$$\sim(1 \ll 5) =$$

1	1	0	1	1	1	1	1
---	---	---	---	---	---	---	---

Deslocamento de bits

- Usando deslocamento bits para criação de máscara

Representação na memória:



AND
bitwise

Deslocamento de bits

- A operação de deslocamento de bits à esquerda é também uma operação multiplicação por um valor em potência de 2.
- Exemplo 1:

3 =

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

3 << 1 =

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

= $3 \cdot 2^1 = 6$

Deslocamento de bits

- A operação de deslocamento de bits à esquerda é também uma operação multiplicação por um valor em potência de 2.
- Exemplo 1:

3 =

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

3 << 2 =

0	0	0	0	1	1	0	0
---	---	---	---	---	---	---	---

= $3 \cdot 2^2 = 12$

Deslocamento de bits

- A operação de deslocamento de bits à esquerda é também uma operação multiplicação por um valor em potência de 2.
- Exemplo 1:

3 =

0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---

3 << 3 =

0	0	0	1	1	0	0	0
---	---	---	---	---	---	---	---

= $3 \cdot 2^3 = 24$

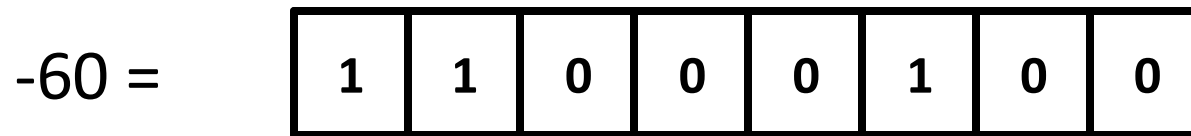
Deslocamento de bits

- O operador de deslocamento para a direita (>>) move os bits de um valor para a direita por um número especificado de posições.
- Os bits "deslocados" para fora da extremidade direita são descartados.
- O preenchimento dos novos bits à esquerda depende se o valor é com sinal (arithmetic shift) ou sem sinal (logical shift).
- Em valores sem sinal, os novos bits são preenchidos com zeros. Em valores com sinal, o bit mais significativo (bit de sinal) é usado para preencher os novos bits (mantendo o sinal do número).

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

`int8_t` x = -60 ;



Deslocamento de bits

- Exemplo de deslocamento à esquerda:

`int8_t` x = -60 ;

$$-60 \gg 1 = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ \hline \end{array} = -60 / 2^1 = -30$$

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

`int8_t` x = -60 ;

$$-60 \gg 2 = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ \hline \end{array} = -60 / 2^2 = -15$$

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

`int8_t` x = -60 ;

$$-60 \gg 3 = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ \hline \end{array} = -60 / 2^3 = -8$$

Deslocamento de bits

- Exemplo de deslocamento à esquerda:

`int8_t` x = -60 ;

$$-60 \gg 4 = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ \hline \end{array} = -60 / 2^4 = -4$$